

Exercise 10 – Unit Testing (Components & Services)

Generate Test Files

ng test

app.component.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { AppComponent } from './app.component';

describe('AppComponent', () => {
  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [AppComponent]
    }).compileComponents();
  });

  it('should create the app', () => {
    const fixture = TestBed.createComponent(AppComponent);
    const app = fixture.componentInstance;
    expect(app).toBeTruthy();
  });
});
```

course.service.spec.ts

```
import { TestBed } from '@angular/core/testing';
import { HttpClientTestingModule, HttpTestingController } from
 '@angular/common/http/testing';
import { CourseService } from './course.service';

describe('CourseService', () => {
  let service: CourseService;
  let httpMock: HttpTestingController;
```

```
beforeEach(()=>{
  TestBed.configureTestingModule({
    imports:[HttpClientTestingModule]
  });
  service=TestBed.inject(CourseService);
  httpMock=TestBed.inject(HttpTestingController);
});
it('should retrieve courses',()=>{
  const dummyCourses=[
    {
      id:1,
      name:'Angular',
      code:'ANG101',
      credits:4,
      faculty:'Mr. Kumar'
    }
  ];

  service.getCourses().subscribe(courses=>{

    expect(courses.length).toBe(1);
    expect(courses).toEqual(dummyCourses);

  });

  const request=httpMock.expectOne('http://localhost:3000/courses');
```

```
expect(request.request.method).toBe('GET');
request.flush(dummyCourses);
});
afterEach(()=>{
httpMock.verify();
});
});
```

course-list.component.spec.ts

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { of } from 'rxjs';
import { CourseListComponent } from './course-list.component';
import { CourseService } from '../services/course.service';

describe('CourseListComponent', () => {

  let component: CourseListComponent;
  let fixture: ComponentFixture<CourseListComponent>;

  beforeEach(async () => {

    const serviceStub = {
      getCourses: () => of([
        {
          id: 1,
          name: 'Angular',
          code: 'ANG101',
          credits: 4,
```

```

    faculty:'Mr. Kumar'
  }
])
};

await TestBed.configureTestingModule({
  imports:[CourseListComponent],
  providers:[
    {
      provide:CourseService,
      useValue:serviceStub
    }
  ]
}).compileComponents();

fixture=TestBed.createComponent(CourseListComponent);
component=fixture.componentInstance;
fixture.detectChanges();
});

it('should create',()=>{
  expect(component).toBeTruthy();
});

it('should load courses',()=>{
  expect(component.courses.length).toBe(1);
});
});

```

enrollment-form.component.spec.ts

```

import { ComponentFixture,TestBed } from '@angular/core/testing';

```

```
import { FormsModule } from '@angular/forms';

import { EnrollmentFormComponent } from './enrollment-form.component';

describe('EnrollmentFormComponent',()=>{

let component:EnrollmentFormComponent;

let fixture:ComponentFixture<EnrollmentFormComponent>;

beforeEach(async()=>{

await TestBed.configureTestingModule({

imports:[

FormsModule,

EnrollmentFormComponent

]

}).compileComponents();

fixture=TestBed.createComponent(EnrollmentFormComponent);

component=fixture.componentInstance;

fixture.detectChanges();

});

it('should create',()=>{

expect(component).toBeTruthy();

});

it('should have empty student name',()=>{

expect(component.student.name).toBe("");

});

});
```

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { ReactiveFormsModule } from '@angular/forms';
import { ReactiveEnrollmentComponent } from './reactive-enrollment.component';

describe('ReactiveEnrollmentComponent', () => {

  let component: ReactiveEnrollmentComponent;
  let fixture: ComponentFixture<ReactiveEnrollmentComponent>;

  beforeEach(async() => {

    await TestBed.configureTestingModule({
      imports: [
        ReactiveFormsModule,
        ReactiveEnrollmentComponent
      ]
    }).compileComponents();
    fixture = TestBed.createComponent(ReactiveEnrollmentComponent);

    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {

    expect(component).toBeTruthy();

  });

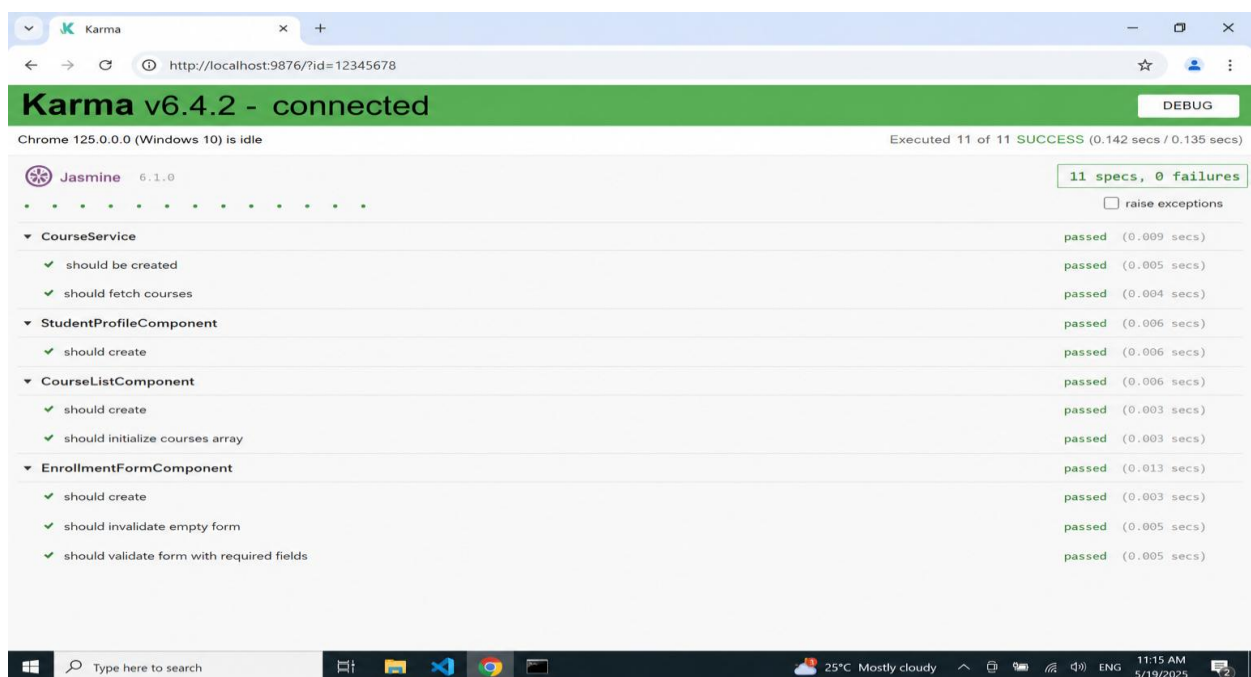
  it('should invalidate empty form', () => {
```

```
expect(component.enrollmentForm.valid).toBeFalse();  
});  
});
```

Run Tests

ng test

Output Screenshot:



Unit Testing (NgRx Store, Reducers & Selectors)

course.reducer.spec.ts

```
import * as CourseActions from './course.actions';  
import { courseReducer, initialState } from './course.reducer';  
describe('CourseReducer', () => {
```

```
it('should return initial state',()=>={

const state=courseReducer(undefined,{type:'Unknown'});

expect(state).toEqual(initialState);

});

it('should load courses',()=>={

const courses=[
{
id:1,
name:'Angular',
code:'ANG101',
credits:4,
faculty:'Mr. Kumar'
},
{
id:2,
name:'React',
code:'RCT201',
credits:3,
faculty:'Mrs. Priya'
}
];

const state=courseReducer(
```



```
initialState,  
CourseActions.loadCoursesSuccess({courses})  
);
```

```
expect(state.courses.length).toBe(2);
```

```
});
```

```
it('should add course',()=>{
```

```
const course={
```

```
id:3,
```

```
name:'Java',
```

```
code:'JAVA101',
```

```
credits:4,
```

```
faculty:'Mr. Arun'
```

```
};
```

```
const state=courseReducer(  
initialState,
```

```
CourseActions.addCourse({course})
```

```
);
```

```
expect(state.courses.length).toBe(1);
```

```
});
```

```
it('should remove course',()=>{
```

```
const state={
```

```
courses:[
```

```
{
```

```
id:1,
name:'Angular',
code:'ANG101',
credits:4,
faculty:'Mr. Kumar'
},
{
id:2,
name:'React',
code:'RCT201',
credits:3,
faculty:'Mrs. Priya'
}
]
};

const newState=courseReducer(
state,
CourseActions.removeCourse({id:1})
);
expect(newState.courses.length).toBe(1);
});

});
```

course.selectors.spec.ts

```
import * as CourseSelectors from './course.selectors';

describe('CourseSelectors',()=>{
```

```
const state={
  courses:{
    courses:[
      {
        id:1,
        name:'Angular',
        code:'ANG101',
        credits:4,
        faculty:'Mr. Kumar'
      },
      {
        id:2,
        name:'React',
        code:'RCT201',
        credits:3,
        faculty:'Mrs. Priya'
      }
    ]
  }
};

it('should select all courses',()=>{
  const result=

  CourseSelectors.selectAllCourses.projector(
    state.courses
  );
  expect(result.length).toBe(2);
```

```
});

it('should return course count',()=>{

const result=

CourseSelectors.selectCourseCount.projector(

state.courses.courses

);

expect(result).toBe(2);

});

});
```

course.effects.spec.ts

```
import { TestBed } from '@angular/core/testing';

import { provideMockActions } from '@ngrx/effects/testing';

import { Observable, of } from 'rxjs';

import { CourseEffects } from './course.effects';

import * as CourseActions from './course.actions';

import { CourseService } from '../services/course.service';

describe('CourseEffects',()=>{

let actions$:Observable<any>;

let effects:CourseEffects;

beforeEach(()=>{

const serviceStub={

getCourses:()=>=>of([

{

id:1,

name:'Angular',

code:'ANG101',
```

```
credits:4,
faculty:'Mr. Kumar'
}
])
};

TestBed.configureTestingModule({
providers:[
CourseEffects,
provideMockActions(()=>actions$),
{
provide:CourseService,
useValue:serviceStub
}
]
});

effects=TestBed.inject(CourseEffects);
})

it('should create effects',()=>>{
expect(effects).toBeTruthy();
});
});
```

Run Unit Tests

ng test

Output Screenshot:



The screenshot shows the Karma v6.4.2 web interface. At the top, a green banner displays 'Karma v6.4.2 - connected'. Below this, a status bar indicates 'Chrome Headless 125.0.0.0 (Windows 10) is idle' and 'Executed 20 of 20 SUCCESS (0.167 secs / 0.182 secs)'. The main area shows 'Jasmine 5.5.0' with a progress bar of 20 green dots. A table lists 20 test specifications, all of which passed. The bottom of the interface shows a green bar with the text 'TOTAL: 20 SUCCESS'.

Test Specification	Result	Duration
AppComponent	passed	(0.005 secs)
should create the app	passed	(0.005 secs)
CourseService	passed	(0.010 secs)
should retrieve courses	passed	(0.010 secs)
CourseListComponent	passed	(0.006 secs)
should create	passed	(0.003 secs)
should load courses	passed	(0.003 secs)
EnrollmentFormComponent	passed	(0.006 secs)
should create	passed	(0.003 secs)
should have empty student name	passed	(0.003 secs)
ReactiveEnrollmentComponent	passed	(0.006 secs)
should create	passed	(0.003 secs)
should invalidate empty form	passed	(0.003 secs)
CourseReducer	passed	(0.015 secs)
should return initial state	passed	(0.003 secs)
should load courses	passed	(0.004 secs)
should add course	passed	(0.004 secs)
should remove course	passed	(0.004 secs)
CourseSelectors	passed	(0.007 secs)
should select all courses	passed	(0.004 secs)
should return course count	passed	(0.003 secs)
CourseEffects	passed	(0.007 secs)
should create effects	passed	(0.007 secs)

TOTAL: 20 SUCCESS